DAY 01 · BEGINNER · FREE API

Your First AI Program

Ten lines of Python that ask a model a question, and an honest look at what is happening on the other end.

`ChatOpenRouter` `.invoke()` `temperature` `:free model` `Notebook`

KGP Talkie
LEARN. BUILD. TALK AI

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

The 10-Day Course Map

WEEK 1 TALKING TO A MODEL

01 — Your First AI Program (YOU ARE HERE)
Ask a model, print the answer

02 — Roles and Messages
The system message sets the job

03 — Prompt Templates
One template, many inputs

YOU WILL HAVE BUILT
a script that answers anything you type into it

WEEK 2 BUILDING SOMETHING

04 — Chains: The Pipe
`prompt | model | parser`

05 — Structured Output
An object, not a paragraph

06 — Memory
A checkpointer and a thread id

YOU WILL HAVE BUILT
a chatbot that remembers what you said a minute ago

WEEK 3 GIVING IT ABILITIES

07 — Tools
Your Python, called by the model

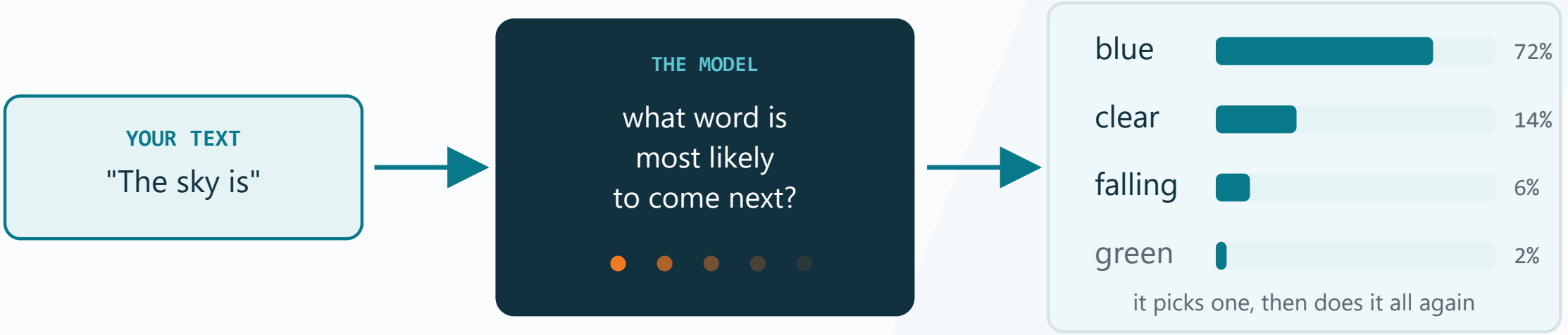
08 — Your First Agent
It decides when to call them

09 — Chunks and Vectors
Find a paragraph by meaning

10 — RAG with RAGWire
Citations, and a refusal

One notebook and one deck a day, **60 to 90 minutes** each. Every model used is a **free** one, and Docker only turns up on Day 9.

How a Language Model Works



A language model has no facts stored in it. It has **probabilities**. Everything it says is the result of picking a likely next word, thousands of times in a row.

Tokens: What the Model Reads



Text is chopped into **tokens** before the model sees it. A long word can be several tokens, and a token is roughly four characters of English.

Why Models Hallucinate

WHAT IT IS TRAINED TO DO

Sound like fluent English



Measured, scored and improved
on billions of examples.

WHAT IT IS NOT TRAINED TO DO

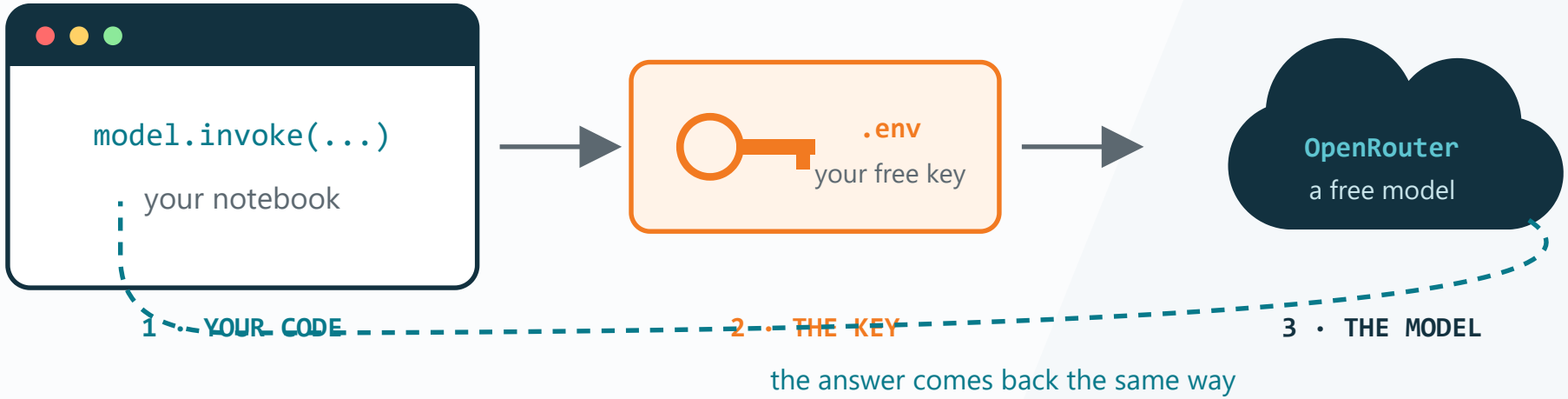
Check whether it is right

is this a real fact?	not scored
is this source real?	not scored
do I actually know?	not scored

A wrong answer and a right answer
are written in the same voice.

Fluency is trained. Truth is not. This is the single most useful thing to know about a language model, and everything from Day 7 onward is a way of working around it.

The Setup: Code, Key and Model



You never touch the model. You touch a **connection** to it, and the key proves you are allowed to.

ONE FILE, NEVER IN CODE, NEVER COMMITTED

Keep the API Key in .env

06

KGP Talkie
LEARN. BUILD. TALK AI

THE ONLY PATH THE KEY TAKES



WHERE IT MUST NEVER GO

~~a notebook cell~~
it gets saved with the output

~~a print() call~~
screen shares and screenshots

~~a git commit~~
bots scan public repos

`.env` is already in `.gitignore`. Leave it there. A leaked free key is still your key, and revoking one is a worse afternoon than setting one up.

Your First Model Call


```
from dotenv import load_dotenv
          from langchain_openrouter import ChatOpenRo

          load_dotenv()

          model = ChatOpenRouter(
          model="nvidia/nemotron-3.5-lightning:free"
          )

          reply = model.invoke("What is LangChain?")

          reply.text
```

> **:free** at the end of the model name is what keeps this free.

WHAT EACH LINE DOES

- **load_dotenv()** reads your key out of `.env` so you never type it.
- **ChatOpenRouter(...)** dials the number. Nothing is sent yet.
- **.invoke(...)** sends the text and waits for the reply.
- **.text** is the answer. The reply also carries token counts.

The Reply Is an Object

reply

AIMessage

.text
the answer you were expecting

.usage_metadata
tokens in, tokens out, total

.response_metadata
which model actually answered

one object, three useful parts

WHY .text AND NOT print(reply)

Printing the whole object gives you the answer buried in metadata. In a notebook, just let the last line show it.

WHY THE TOKEN COUNTS MATTER

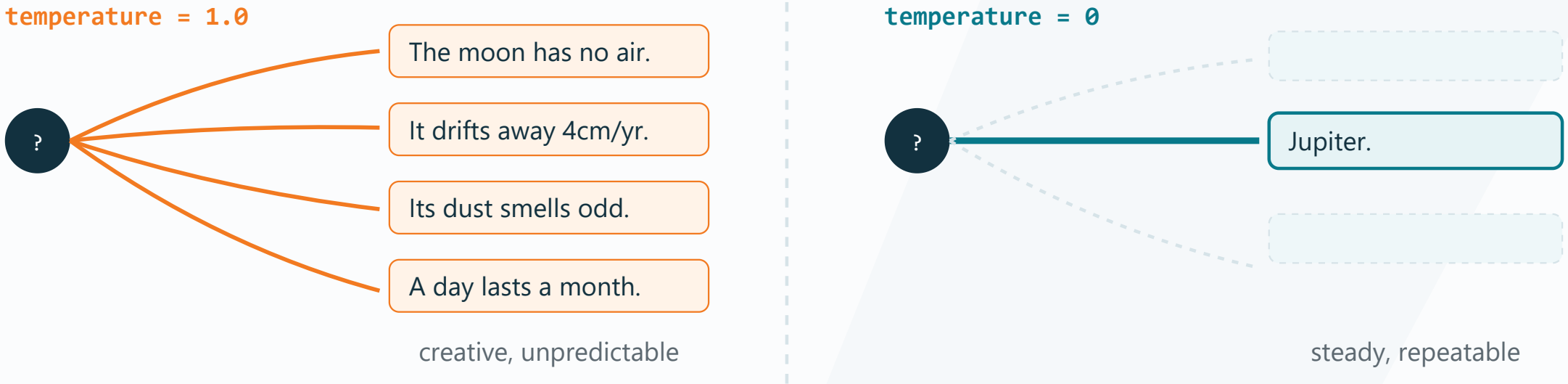
On a free model they cost nothing, but they are the same numbers you would pay for later. Watch them grow.

WHY THE MODEL NAME MATTERS

A free model can be swapped or retired. If the answers change overnight, this field is where you look first.

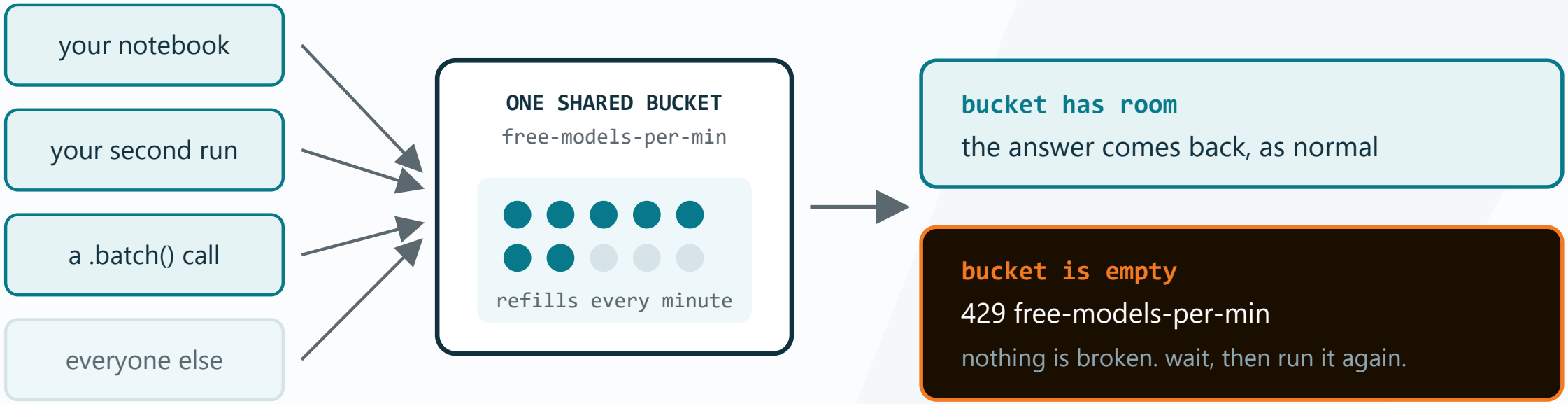
The reply is an **object**, not a string. Reading the other two fields is the difference between using an API and understanding it.

Temperature Controls Randomness



Ask twice, get two answers. That is not a bug. Use **temperature=0** when you want a fact, and leave it high when you want ideas.

Rate Limits: the 429



The limit is per minute and it is **shared**. Three notebooks in a row will trip it, and so will a room of twenty people running the same cell at once.

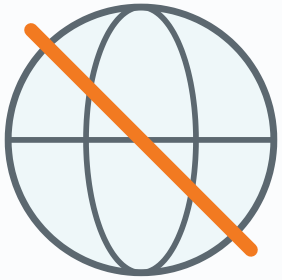
Three Things It Cannot Do



No clock

It does not know today's date

`fixed on Day 7`



No documents

It has never read a single file of yours

`fixed on Day 10`



No memory

Every call starts completely empty

`fixed on Day 6`

None of these are flaws to be fixed by a better prompt. Each one is a missing **connection to the world**, and the rest of this course is about wiring them in.